

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра радіофізики та комп'ютерних технологій

Звіт

З лабораторної №1

З дисципліни «Професійний Python»

На тему «**Організація професійного Python-проєкту та Git**
Варіант: 4 (Система банківських рахунків)»

Виконав:

Студент 3 курсу

групи ФЕП-31

Войтович М.А.

Перевірив:

проф. Свелеба С. А.

Львів 2026

Мета: Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки (venv), формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл конфігурації pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту в README.md.

Теоретичні відомості

src-layout: Архітектурний підхід, де вихідний код розміщується в каталозі src/<package_name>/. Це запобігає випадковому імпорту незавершеного коду з поточного робочого каталогу та гарантує виконання коду через встановлений пакет.

Віртуальне середовище (venv): Ізольоване середовище Python, яке дозволяє встановлювати залежності окремо для кожного проєкту, не засмічуючи системну версію Python.

Модулі та пакети: Модуль — це окремий .py файл. Пакет — це каталог, який містить файл __init__.py та інші модулі.

Точка входу: Конструкція if __name__ == "__main__": гарантує, що головна функція виконується лише при прямому запуску файла як скрипта, а не при його імпорті іншими модулями.

pyproject.toml: Сучасний стандарт метаданих та конфігурації збірки Python-проєктів (згідно з PEP 517 / PEP 621).

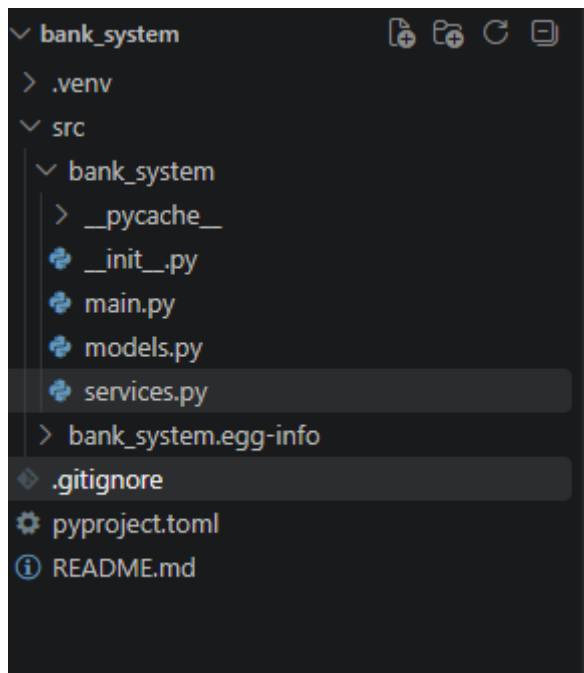
Git: Розподілена система контролю версій, яка дозволяє фіксувати зміни (commit), відстежувати стан файлів (status) та переглядати історію розробки (log).

Постановка задачі

Створити консольний Python-застосунок з використанням src-layout, який відповідає стандартам PEP 8, містить строкову типізацію (Type Hints), розділений на декілька модулів (модель даних, бізнес-логіка, точка входу), має конфігураційні файли pyproject.toml, .gitignore, файл документації README.md та збережений у репозиторії Git з логічними комітами.

Хід виконання

1. Створено структуру каталогу за принципом src-layout:



2. Налаштування віртуального середовища:

```
PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> python -m venv .venv
PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> .venv\Scripts\activate
```

3. Розробка модулів:

src/bank_system/models.py: Визначено dataclass BankAccount для збереження даних рахунку та реалізовано методи deposit() і withdraw() із перевіркою на від'ємний баланс.

```
1  from dataclasses import dataclass
2
3
4  @dataclass
5  class BankAccount:
6      """Модель банківського рахунку.
7
8      Забороняє встановлення або зміну балансу до від'ємного значення.
9      """
10     account_number: str
11     owner_name: str
12     balance: float = 0.0
13
14     def deposit(self, amount: float) -> None:
15         """Поповнення рахунку."""
16         if amount <= 0:
17             raise ValueError("Сума поповнення повинна бути більшою за нуль.")
18         self.balance += amount
19
20     def withdraw(self, amount: float) -> None:
21         """Списання коштів з рахунку."""
22         if amount <= 0:
23             raise ValueError("Сума списання повинна бути більшою за нуль.")
24         if amount > self.balance:
25             raise ValueError(
26                 f"Недостатньо коштів! Поточний баланс: {self.balance:.2f} грн. "
27                 "Від'ємний баланс заборонено."
28             )
29         self.balance -= amount
```

src/bank_system/services.py: Створено клас BankService для виконання операцій пошуку рахунку, відкриття нових рахунків та підрахунку загального капіталу.

```

from typing import List, Optional
from bank_system.models import BankAccount

class BankService:
    """Сервіс для управління групою банківських рахунків."""

    def __init__(self) -> None:
        self.accounts: List[BankAccount] = []

    def create_account(
        self, account_number: str, owner_name: str, initial_balance: float = 0.0
    ) -> BankAccount:
        """Створення нового рахунку."""
        if self.find_account_by_number(account_number) is not None:
            raise ValueError(f"Рахунок з номером '{account_number}' вже існує.")
        if initial_balance < 0:
            raise ValueError("Початковий баланс не може бути від'ємним.")

        account = BankAccount(
            account_number=account_number,
            owner_name=owner_name,
            balance=initial_balance
        )
        self.accounts.append(account)
        return account

    def find_account_by_number(self, account_number: str) -> Optional[BankAccount]:
        """Пошук рахунку за точним номером."""
        for account in self.accounts:
            if account.account_number == account_number:
                return account
        return None

    def search_accounts_by_owner(self, owner_name: str) -> List[BankAccount]:
        """Пошук рахунків за ім'ям власника (підстрока)."""
        query = owner_name.strip().lower()
        return [
            acc for acc in self.accounts
            if query in acc.owner_name.lower()
        ]

    def get_total_funds(self) -> float:
        """Обчислення загальної суми коштів на всіх рахунках."""
        return sum(account.balance for account in self.accounts)

```

src/bank_system/main.py: Реалізовано консольне меню користувача та точку входу if __name__ == "__main__":

```

import sys
from bank_system.services import BankService

def display_menu() -> None:
    print("\n=== СИСТЕМА БАНКІВСЬКИХ РАХУНКІВ ===")
    print("1. Створити новий рахунок")
    print("2. Поповнити рахунок")
    print("3. Списати кошти з рахунку")
    print("4. Перевірити баланс рахунку")
    print("5. Пошук рахунку за номером або ім'ям")
    print("6. Показати загальну суму коштів у банку")
    print("7. Вийти з програми")

def main() -> None:
    bank_service = BankService()
    bank_service.create_account("UA1001", "Шевченко Тарас", 1500.0)
    bank_service.create_account("UA1002", "Українка Леся", 3200.5)

    while True:
        display_menu()
        choice = input("Оберіть дію (1-7): ").strip()

        if choice == "1":
            acc_num = input("Введіть номер рахунку (наприклад, UA1003): ").strip()
            owner = input("Введіть ПІБ власника: ").strip()
            try:
                initial_bal = float(input("Введіть початковий баланс (грн): "))
                acc = bank_service.create_account(acc_num, owner, initial_bal)
                print(f"Успішно створено рахунок {acc.account_number} для {acc.owner_name}!")
            except ValueError as err:
                print(f"Помилка створення: {err}")

        elif choice == "2":
            acc_num = input("Введіть номер рахунку: ").strip()
            acc = bank_service.find_account_by_number(acc_num)
            if not acc:
                print("Рахунок не знайдено.")
                continue
            try:
                amount = float(input("Введіть суму поповнення (грн): "))
                acc.deposit(amount)
                print(f"Рахунок поповнено! Поточний баланс: {acc.balance:.2f} грн.")
            except ValueError as err:
                print(f"Помилка: {err}")

        elif choice == "3":
            acc_num = input("Введіть номер рахунку: ").strip()
            acc = bank_service.find_account_by_number(acc_num)
            if not acc:

```

```

        if not acc:
            print("Рахунок не знайдено.")
            continue

        try:
            amount = float(input("Введіть суму списання (грн): "))
            acc.withdraw(amount)
            print(f"Списання успішне! Новий баланс: {acc.balance:.2f} грн.")
        except ValueError as err:
            print(f"Помилка: {err}")

    elif choice == "4":
        acc_num = input("Введіть номер рахунку: ").strip()
        acc = bank_service.find_account_by_number(acc_num)
        if acc:
            print(f"Рахунок: {acc.account_number} | Власник: {acc.owner_name} | Баланс: {acc.balance:.2f} грн.")
        else:
            print("Рахунок не знайдено.")

    elif choice == "5":
        query = input("Введіть номер рахунку або ім'я клієнта для пошуку: ").strip()
        by_num = bank_service.find_account_by_number(query)
        by_name = bank_service.search_accounts_by_owner(query)

        found_accounts = []
        if by_num:
            found_accounts.append(by_num)
        for acc in by_name:
            if acc not in found_accounts:
                found_accounts.append(acc)

        if found_accounts:
            print("\nЗнайдені рахунки:")
            for acc in found_accounts:
                print(f"- {acc.account_number} ({acc.owner_name}): {acc.balance:.2f} грн.")
        else:
            print("Рахунків за вашим запитом не знайдено.")

    elif choice == "6":
        total = bank_service.get_total_funds()
        print(f"\nЗагальна сума коштів усіх клієнтів у банку: {total:.2f} грн.")

    elif choice == "7":
        print("Завершення роботи системи. До побачення!")
        sys.exit(0)

    else:
        print("Некоректний вибір. Спробуйте ще раз.")

if __name__ == "__main__":
    main()

```

4. Створення конфігураційних файлів:

Додано `pyproject.toml` для налаштування збірки проєкту за допомогою `setuptools`.
 Налаштовано `.gitignore` для виключення папки `.venv/`, `caches` та тимчасових файлів IDE.

`pyproject.toml`

```

[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "bank-system"
version = "0.1.0"
description = "Система банківських рахунків для лабораторної роботи №1"
readme = "README.md"
requires-python = ">=3.11"
dependencies = []

[tool.setuptools.packages.find]
where = ["src"]

```

`.gitignore`

```

❖ .gitignore
1  .venv/
2  venv/
3  ENV/
4
5  __pycache__/
6  *.py[cod]
7  *$py.class
8
9  build/
10 devel/
11 dist/
12 downloads/
13 eggs/
14 .eggs/
15 *.egg-info/
16
17 .idea/
18 .vscode/
19
20 .pytest_cache/
21 .coverage

```

Оформлено README.md та Git з інструкціями щодо встановлення та запуску.

5. Інсталяція та перевірка роботи:

Проект встановлено в режимі редагування

```

(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> python -m pip install -e .
Obtaining file:///C:/Users/Maximus/Desktop/Професійний пітон\bank_system
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: bank-system
Building editable for bank-system (pyproject.toml) ... done
Created wheel for bank-system: filename=bank_system-0.1.0-0.editable-py3-none-any.whl size=1601 sha256=9e3b8ff40f43f5dee91949206c6826606dc0e28be1f033e4b9d11cfcf206a8b
Stored in directory: C:\Users\Maximus\AppData\Local\Temp\pip-ephem-wheel-cache-dn3c2xql\wheels\61\46\97\2b239afe9ff9d3419015b653544617174fc0113efc78037757
Successfully built bank-system
Installing collected packages: bank-system
Successfully installed bank-system-0.1.0

```

Запуск виконано як модуль Python

```

(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> python -m bank_system.main

```

README.md

```

1  ∨ # Bank Account System
2  Навчальний проєкт з дисципліни «Професійний Python».
3
4  ∨ ## Description
5  Консольний застосунок для керування банківськими рахунками (Варіант 4).
6  Реалізовано: поповнення, списання, пошук та загальний підрахунок коштів.
7
8  ∨ ## Requirements
9  Python 3.11+
10
11 ∨ ## Installation
12 ∨ ```bash
13 python -m venv .venv
14 # Для Windows: .venv\Scripts\activate
15 python -m pip install -e .

```

Git status

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add .gitignore pyproject.toml src/bank_system/_init_.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   pyproject.toml
        new file:   src/bank_system/_init_.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        src/bank_system/main.py
        src/bank_system/models.py
        src/bank_system/services.py

(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Створення структури проекту"
[main (root-commit) 454af57] Створення структури проекту
 3 files changed, 37 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 pyproject.toml
 create mode 100644 src/bank_system/_init_.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/models.py src/bank_system/services.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git status
On branch main

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   src/bank_system/models.py
        new file:   src/bank_system/services.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        src/bank_system/main.py

(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано модель банківського рахунку та бізнес-логіку"
[main 603e965] Додано модель банківського рахунку та бізнес-логіку
 2 files changed, 74 insertions(+)
 create mode 100644 src/bank_system/models.py
 create mode 100644 src/bank_system/services.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/main.py README.md
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано точку входу та документацію застосунку Bank System"
[main 53d3b57] Додано точку входу та документацію застосунку Bank System
 2 files changed, 180 insertions(+)
 create mode 100644 README.md
 create mode 100644 src/bank_system/main.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git status
```

git log –online

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git log --online
53d3b57 (HEAD -> main) Додано точку входу та документацію застосунку Bank System
603e965 Додано модель банківського рахунку та бізнес-логіку
454af57 Створення структури проекту
```

Індивідуальне завдання (Варіант 4)

Реалізувати систему банківських рахунків зі збереженням ім'я клієнта, номера рахунку та балансу.

Вимоги та їх реалізація:

Збереження даних: Модель BankAccount зберігає account_number: str, owner_name: str, balance: float.

Поповнення та списання: Реалізовано через методи deposit(amount) та withdraw(amount).

Заборона від'ємного балансу: Метод withdraw() викликає виключення ValueError, якщо сума списання перевищує поточний баланс, або якщо сума ≤ 0 .

Перевірка балансу та пошук: Метод find_account_by_number() здійснює точний пошук, а search_accounts_by_owner() — підстроковий пошук за ім'ям.

Обчислення загальної суми: Метод get_total_funds() обчислює суму коштів на всіх рахунках системи.

Експериментальна частина

Тестування роботи програми та обробки помилок:

Сценарій 1: Створення рахунків та виведення початкового стану

```
=== СИСТЕМА БАНКІВСЬКИХ РАХУНКІВ ===
1. Створити новий рахунок
2. Поповнити рахунок
3. Списати кошти з рахунку
4. Перевірити баланс рахунку
5. Пошук рахунку за номером або ім'ям
6. Показати загальну суму коштів у банку
7. Вийти з програми
Оберіть дію (1-7): 1
Введіть номер рахунку (наприклад, UA1003): UA1005
Введіть ПІБ власника: Maksym Voitovych
Введіть початковий баланс (грн): 100
Успішно створено рахунок UA1005 для Maksym Voitovych!
```

```
=== СИСТЕМА БАНКІВСЬКИХ РАХУНКІВ ===
1. Створити новий рахунок
2. Поповнити рахунок
3. Списати кошти з рахунку
4. Перевірити баланс рахунку
5. Пошук рахунку за номером або ім'ям
6. Показати загальну суму коштів у банку
7. Вийти з програми
Оберіть дію (1-7): 4
Введіть номер рахунку: UA1005
Рахунок: UA1005 | Власник: Maksym Voitovych | Баланс: 100.00 грн.
```

Сценарій 2: Спроба списати суму, більшу за баланс (захист від від'ємного балансу)

```
=== СИСТЕМА БАНКІВСЬКИХ РАХУНКІВ ===
1. Створити новий рахунок
2. Поповнити рахунок
3. Списати кошти з рахунку
4. Перевірити баланс рахунку
5. Пошук рахунку за номером або ім'ям
6. Показати загальну суму коштів у банку
7. Вийти з програми
Оберіть дію (1-7): 3
Введіть номер рахунку: UA1005
Введіть суму списання (грн): 200
Помилка: Недостатньо коштів! Поточний баланс: 100.00 грн. Від'ємний баланс заборонено.
```

Сценарій 3: Поповнення рахунку та перевірка загальної суми коштів

```

=== СИСТЕМА БАНКІВСЬКИХ РАХУНКІВ ===
1. Створити новий рахунок
2. Поповнити рахунок
3. Списати кошти з рахунку
4. Перевірити баланс рахунку
5. Пошук рахунку за номером або ім'ям
6. Показати загальну суму коштів у банку
7. Вийти з програми
Оберіть дію (1-7): 2
Введіть номер рахунку: UA1005
Введіть суму поповнення (грн): 1000
Рахунок поповнено! Поточний баланс: 1100.00 грн.

=== СИСТЕМА БАНКІВСЬКИХ РАХУНКІВ ===
1. Створити новий рахунок
2. Поповнити рахунок
3. Списати кошти з рахунку
4. Перевірити баланс рахунку
5. Пошук рахунку за номером або ім'ям
6. Показати загальну суму коштів у банку
7. Вийти з програми
Оберіть дію (1-7): 6

Загальна сума коштів усіх клієнтів у банку: 5800.50 грн.

```

7. Вимоги до коду

Вимога лабораторної роботи	Стан виконання	Докази / Примітка
Використання src-layout	Виконано	Код розміщено у src/bank_system/
Щонайменше 3 Python-модулі	Виконано	main.py, models.py, services.py
Наявність data model	Виконано	Dataclass BankAccount у models.py
Окремий модуль бізнес-логіки	Виконано	Клас BankService у services.py
Наявність type hints	Виконано	Усі функції та методи мають анотації типів
Захист від від'ємного балансу	Виконано	Перевірка в withdraw() та create_account()
Точка входу if __name__ == "__main__":	Виконано	Присутня в main.py
Наявність pyproject.toml, .gitignore, README.md	Виконано	Розміщені в корені проєкту
Дотримання PEP 8	Виконано	Іменування змінних, відступи, docstrings

8. Контрольні запитання

1. Що таке Python-проєкт?

Це набір файлів для розробки, запуску та тестування програми.

2. Чим проєкт відрізняється від одного Python script?

Script — це один .py файл, а проєкт має структуру, модулі, залежності та конфігурацію.

3. Що таке Python module?

Це окремий .py файл із кодом, який можна імпортувати.

4. Що таке Python package?

Це папка, яка містить модулі та об'єднує їх у одну структуру.

5. Для чого потрібний __init__.py?

Для визначення та налаштування Python-пакета.

6. Що таке src-layout?

Це структура, де код програми знаходиться в окремій папці src/.

7. Які переваги має src-layout?

Він розділяє код програми та інші файли проєкту й допомагає уникати помилок під час імпорту.

8. Для чого використовується virtual environment?

Для створення окремого середовища з бібліотеками конкретного проєкту.

9. Як створити .venv?

Командою `python -m venv .venv`.

10. Як активувати virtual environment?

Windows:

`.\.venv\Scripts\activate`

Linux/macOS: `source .venv/bin/activate`.

11. Для чого використовується pip?

Для встановлення, оновлення та видалення Python-бібліотек.

12. Чому рекомендується python -m pip?

Щоб використовувати pip саме для потрібного Python.

13. Для чого потрібний pyproject.toml?

Це основний конфігураційний файл Python-проєкту.

14. Що містить секція [project]?

Назву, версію, опис і залежності проєкту.

15. Що таке editable installation?

Встановлення, при якому зміни в коді одразу доступні без повторного встановлення.

16. Для чого використовується if __name__ == "__main__"?

Щоб код виконувався тільки при прямому запуску файлу.

17. Що таке Git?

Це система контролю версій для відстеження змін у проєкті.

18. Що таке repository?

Це сховище проєкту разом з історією Git.

19. Що таке commit?

Це зафіксований стан проєкту в певний момент.

20. Для чого використовується git status?

Показує поточний стан і зміни у файлах.

21. Для чого git add?

Додає зміни до підготовки для комміту.

22. Для чого git commit?

Фіксує підготовлені зміни в історії Git.

23. Що показує git log?

Показує історію коммітів.

24. Для чого потрібен .gitignore?

Щоб вказати файли, які Git не повинен відстежувати.

25. Чому .venv не потрібно комітити?

Він займає багато місця та залежить від операційної системи.

26. Для чого потрібний README?

Для опису проєкту та інструкцій щодо його запуску.

27. Що таке type hints?

Це підказки щодо типів даних у коді.

28. Для чого програму розділяють на modules?

Щоб поділити код на логічні частини та спростити його використання.

29. Що таке business logic?

Це основні правила та логіка роботи програми.

30. Чому потрібно відокремлювати application entry point від business logic?

Щоб код було простіше тестувати, змінювати та підтримувати.

9. Зміст звіту

У ході лабораторної роботи було створено структурований та ізольований Python-проєкт для банківської системи. Було реалізовано модель банківського рахунку, основну логіку роботи з рахунками та інтерфейс для взаємодії з користувачем.

Також було налаштовано конфігурацію проєкту, додано документацію та .gitignore. Для контролю змін використовувався Git, у якому було створено 3 логічні коміти. Програму перевірено на правильність роботи, обробку введених даних та дотримання обмежень балансу. Код оформлено відповідно до вимог PEP 8 та індивідуального завдання.